“Act as an Expert”: Why Your Prompting is Cargo Cult and How to Fix It

What Happens When You Hit Enter

FUTURE SIMPLE • 14 min read • 2025-09-18

https://ai.plainenglish.io/act-as-an-expert-why-your-prompting-is-cargo-cult-and-how-to-fix-it-e07c90fcac2e

Remember late 2022? “Prompt engineering” still sounded like the next hot tech job, the gateway to the “new tech gold rush.” Everyone dove in headfirst - some out of curiosity, others wanting to understand practically: how does this all work, and how can I leverage it?

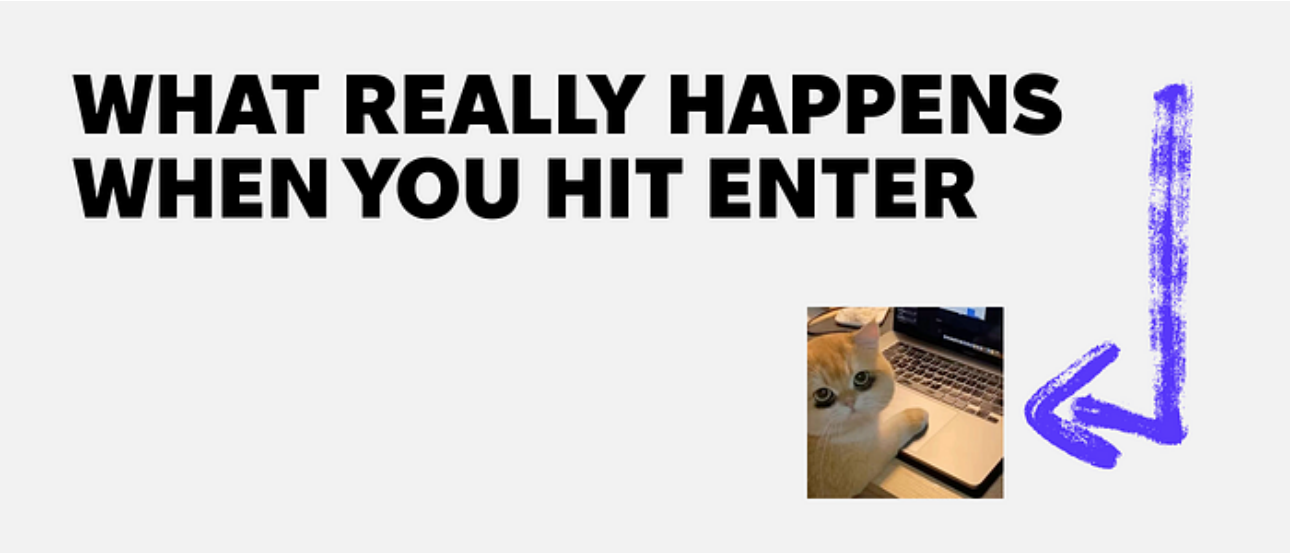
I was in the second camp, though I’d started earlier - reading papers, searching for scientific foundations for making AI work effectively, applying all these “chain-of-thought,” “few-shot,” and other then-obscure incantations in practice.

Long story short, I eventually learned everything I wanted and reached “zen” (or so I thought) - developed my own empirical approaches and stopped obsessively tracking every new publication.

But recently I decided to revisit the topic, to dig deep again - what’s changed, where has academic thinking landed? And to my surprise, the latest research underlying this article shows more or less the same insights I’d reached through trial and error.

The problem is, most people still think prompting is just word games. That there’s some secret phrase that’ll make AI spit out genius. But that’s like trying to operate a nuclear reactor by hitting it with a wrench. Point being: prompt engineering is having an identity crisis, and that’s why I’m writing this.

We’re dealing with one of the most complex systems humanity has created. To use it effectively, you need to understand not its “psychology,” but its architecture, attention mechanisms, and how it actually “sees” information.



What Happens When You Hit Enter

In 90% of cases when someone asks me about prompting, they start with the same question: “How do I write prompts correctly?”. And I,playing the part of the asshole professor, respond with a question: “Do you even understand what we’re about to discuss?” - yes, in my head, I’m casting myself as that toxic-charismatic young professor from a 2000s movie.

So after my question, there’s usually an awkward pause, because it wasn’t rhetorical - it needs an answer.

My point is: people are used to thinking about prompting as a set of rules and tricks. Write “think step by step,” add some examples, wrap it in triple quotes - and it’ll work. But we’re actually dealing with something far more interesting and complex.

Let’s use examples: imagine you’re talking to an entity with trillions neural connections, trained on everything humanity has ever written. This entity doesn’t “understand” language in our sense, but it’s learned to predict the next word so accurately it’s become indistinguishable from understanding. Now think: how would you talk to such an entity?

I’ve been reviewing research with various models lately - short conclusion: everything we thought we knew about prompting two years ago is obsolete. And not because some new, more powerful “life hacks” have emerged, but because the nature of what we’re interacting with has changed.

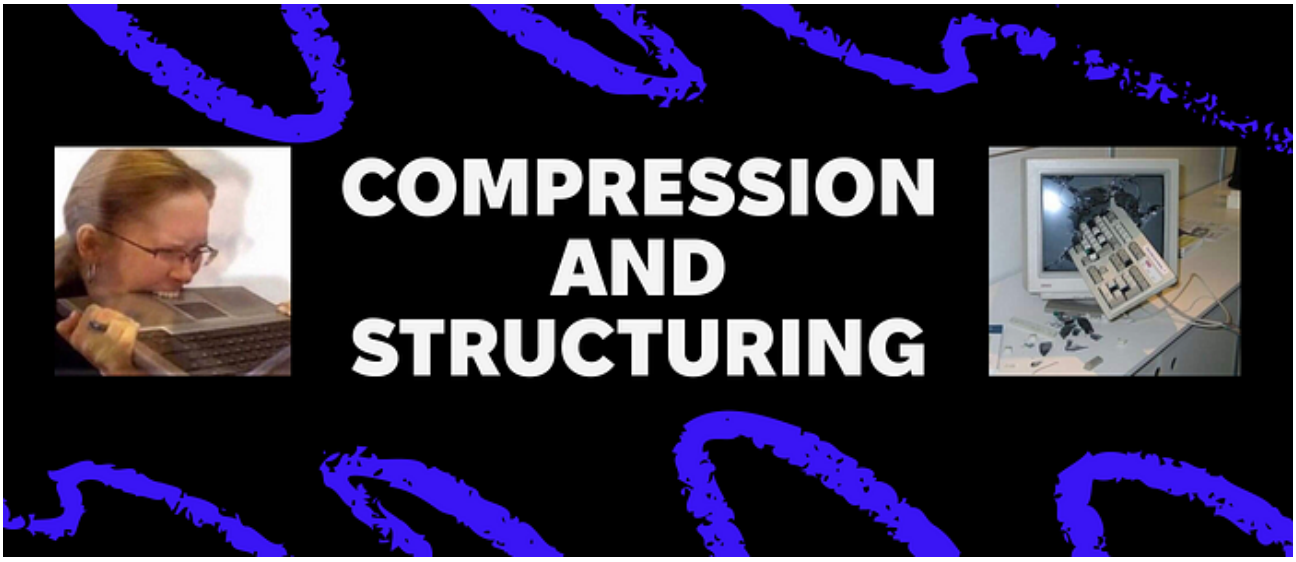
The key insight is how radically attention patterns have changed. In GPT-3-style architectures, attention mechanisms worked rather chaotically. The model could get “stuck” on random tokens, lose the thread due to minor formatting changes. Testing different prompts, you’d often encounter unpredictability - the same query could give radically different results depending on whether you used a period or colon.

Modern models work differently. When I analyze how Claude 4 Sonnet processes a structured prompt, I see attention weights distributing far more logically. The model actually “reads” information hierarchy, understands relationships between text parts, and can maintain context across much greater distances.

The simplest way to see this: compare how different architectures react to changing information order in prompts.

Older models demonstrated recency bias - they gave disproportionate weight to information seen last, at the prompt's end.

Modern models show something closer to genuine semantic processing - they analyze the entire context holistically and can extract the most relevant parts regardless of position in text.



Compression and Structuring

But the biggest revolution came with context windows. The jump from 4-8 thousand tokens to 200 thousand, and now to 2 million - this isn't just quantitative change. It's a qualitative leap that changes the concept of model interaction.

Before, prompting was the art of compression - cramming maximum information into minimum tokens. Now it's the art of structuring - organizing massive information arrays so models can work with them effectively. I can load project documentation into Gemini, add piles of code files, and season it with nearly a dozen research papers - and the model processes it as a unified semantic space.

Practically, this means almost the entire "prompt engineering" apparatus - those endless hack guides - has become as obsolete as trying to manually optimize assembly code for modern processors. Modern models don't need tricks - they need clarity.

Today's best prompt isn't one using some magic formula, but one that clearly structures the task according to how transformer architecture works.

And when I see someone still spreading 2023 approaches - those long chains of "act as an expert, step by step, in the style of" - it's a bit sad.

But here's the paradox - as I write this, most academic research still focuses on GPT-4, and not even recent versions. At best, Claude 3.5 Sonnet.

Meanwhile, people are already using Claude 4 Sonnet, working with reasoning models, and getting used to the release of GPT-5.

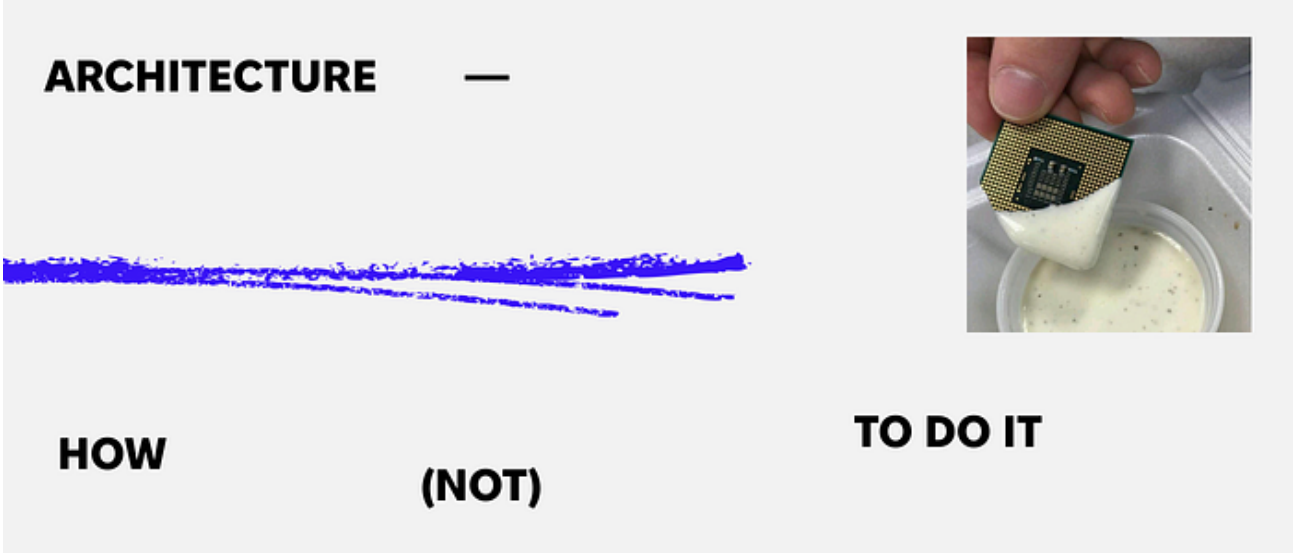
This gap between academia and practice has reached absurd proportions. Scientists publish careful studies about models almost nobody uses anymore. The peer review cycle takes so long that two or three new models have launched. Result: piles of "scientific recommendations" for

models that are practically relics.

Now you'll understand why when I mention GPT-4 and Claude 3.5 research in this article, it's not because they're super relevant, but because systematic studies only exist for them.

Yet whether it's Claude 4, or GPT-5 - they're all built on the same transformer principles. If you understand how attention layers work, how models process semantic structure, how weights distribute between tokens - you can effectively interact with any of these architectures.

You don't need to convince or trick the model. You just need to clearly explain what you want. And if you understand how it works at the architectural level, you can work with it far more effectively than with the next "prompt hacks" from the internet.



Architecture - How (Not) To Do It

Okay, down to business. If you're still writing prompts using old recipes, you risk losing up to half your potential effectiveness. And that's not hyperbole - that's the result of comparing traditional prompting with how modern architectures actually work.

The biggest mistake is blind faith in few-shot examples. If you think "the more examples I give the model, the better it'll understand the task" - here's the counterintuitive truth: for complex tasks, fewer examples often give better results. In other words, quality > quantity.

Why? Modern models have enough context from pre-training to understand tasks from one or two quality examples. Excess examples force the model to focus on surface patterns instead of task semantics - you inadvertently narrow its giant knowledge space to your own limited vision of the task. It's like explaining math through formula memorization instead of understanding principles.

Instead, the real magic comes from structured output. Seriously, if you want to focus on just one thing from this article - focus on structured prompting. Here's how NOT to do it:

Analyze this code and find bugs. Here's the code: [your code here]

And here's how to do it:

```
< a n a l y s i s _ t a s k >  < !  -  T a s k  b l o c k  !'
```

```
<objective>Find logical and syntax errors</objective>
 < c o d e _ s a m p l e >   < !   -   I n p u t   d a t a   b l o c k   !’
[your code here]
</code_sample>
 < o u t p u t _ r e q u i r e m e n t s >   < !   -   O u t p u t   r e q u i r e m e n t
<format>structured_list</format>
<details>
- error_type
- line_number
- explanation
- fix_suggestion
</details>
</output_requirements>
</analysis_task>
```

Before you think “that’s complicated” - it’s actually no harder than writing regular text. Tags are just a way to tell the model “this part is the task, this is data, and this is output requirements.”

How to do this in 30 seconds:

Take any regular prompt and ask yourself three questions:

1. What should the model do? !’ <task> or <objective>

2. What to work with? !’ <input>, <data>, <context>

3. How to present results? !’ <output>, <format>, <requirements>

The difference isn’t just in response quality - it’s in consistency. Structured prompts give more predictable results even when switching between model versions. Why? Because XML creates clear semantic boundaries that attention layers understand intuitively.

But there’s a tricky point - not all formats are created equal. Markdown worked fine before, but modern models handle XML-like markup much better. Not because XML is some “new trend” - it’s because for attention mechanisms, XML tags create clear, nested “containers” of meaning.

The difference is that markdown is visual formatting, while XML is semantic structure. The model doesn’t just “see” text, it “understands” that the <code> token is a child element of <analysis_task>.

Another insight that’ll surprise many: information order in prompts matters, but not how you think. Old models suffered from recency bias - everything at the prompt’s end had disproportionate influence on the response. If you wrote a long prompt with instructions at the beginning, then added “but be concise” at the end, the model would emphasize conciseness.

Modern models demonstrate a different pattern - U-shaped attention. Simply put: they pay most attention to the beginning and end of prompts, while the middle gets less focus. It’s like reading an article - you remember the headline, skim the middle, and focus on conclusions.

Practically this means: place the most important information (task goals, key instructions) at the beginning, details in the middle, and specific format requirements at the end. Not because it’s “correct,” but because that’s, quite literally, how the architecture works.

And please, start moving away from monolithic prompts. Sequential multi-turn conversation

design is becoming not just useful, but sometimes critical. Instead of cramming everything into one query, try breaking complex tasks into logical sequences - like dialogue where each next query builds on the previous response.

Very rough example - instead of: “Analyze this document, find key trends, compare with last year’s data, propose recommendations and create an executive summary”

Better:

1. First: “Analyze this document and identify 5 key trends”

2. Then: “Now compare these trends with last year’s data: [data]”

3. Next: “Based on the comparison, propose 3 specific recommendations”

4. Finally: “Create an executive summary from these recommendations”

Here’s why: modern models maintain context between responses better than they work with huge single prompts.

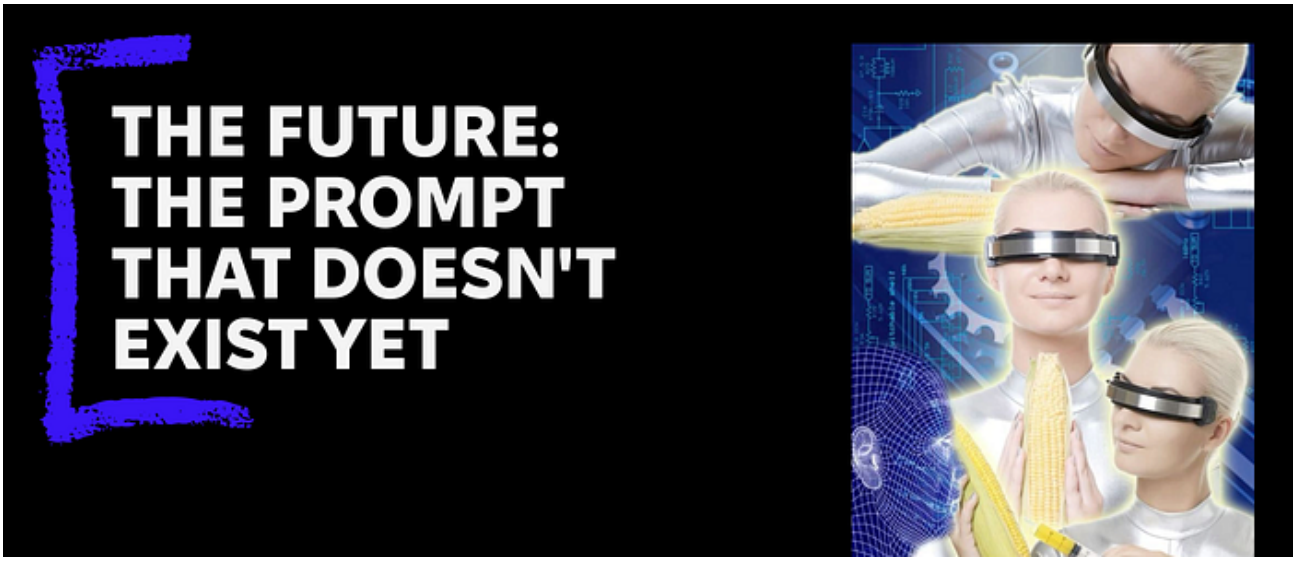
Each new “turn” lets the model recalculate attention weights, focusing on a new, narrower task. Instead of keeping ten goals in “working memory” at once, it sequentially solves one by one, using the previous result as foundation for the next. This is computationally more efficient and much more reliable. Plus, you can adjust direction at any stage.

And probably last but not least - dynamic context management.

When working with large documents (and with 2M context windows that’s now normal), don’t just paste all text into the prompt. Use metadata tags to mark relevant sections:

```
<document_analysis>
<primary_focus>financial_data</primary_focus>
<sections>
<section type="executive_summary" priority="high">
[executive summary text]
</section>
<section type="detailed_analysis" priority="medium">
[detailed analysis]
</section>
<section type="appendix" priority="low">
[appendices]
</section>
</sections>
</document_analysis>
```

This lets the model properly distribute attention between different document parts instead of processing everything as a homogeneous text mass.



The Future: The Prompt That Doesn’t Exist Yet

First, we’re seeing how classic methods like Chain-of-Thought are gradually losing their exclusivity. New models are already trained to think step-by-step by default, and explicit “think step by step” instruction sare yielding diminishing returns.

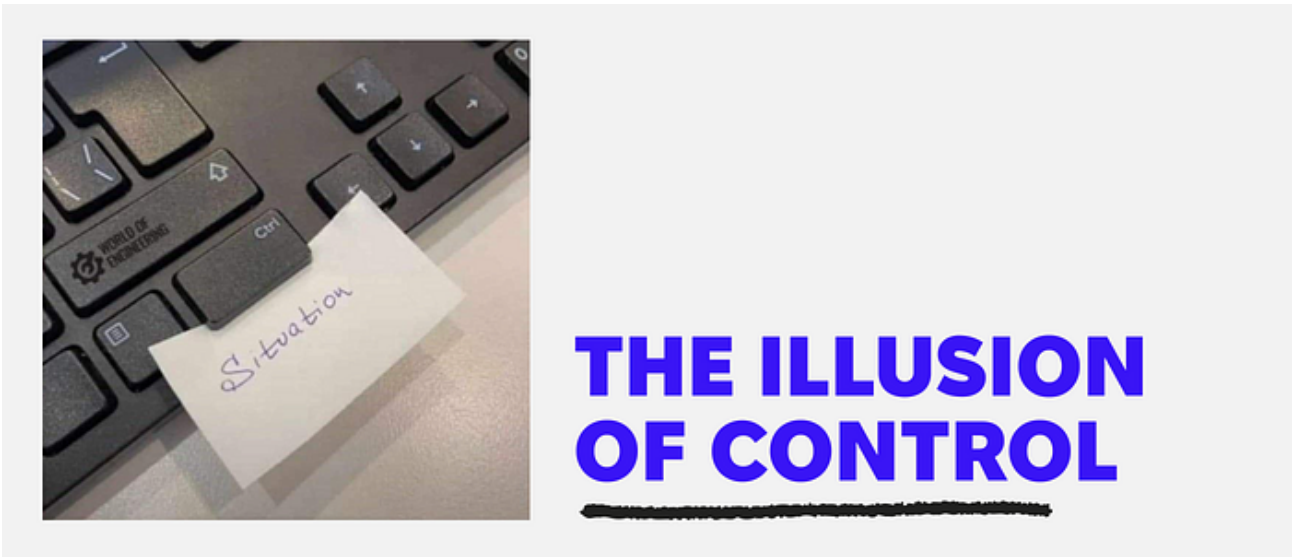
Linear CoT is being replaced by Tree of Thoughts (ToT) and Graph of Thoughts (GoT), which let models not just follow one path, but explore entire trees or even graphs of possible solutions, evaluate them, and discard dead-end branches.

Another development vector is abandoning natural language as the sole tool. On the horizon is the concept of “prompt compression.”

Instead of writing lengthy instructions and examples, we’re learning to compress them into compact, information-dense “soft prompts.” These aren’t natural language anymore, but trained, continuous vectors the model understands much more efficiently. Imagine instead of a long letter, you send a concentrated thought-bundle that instantly unfolds in your correspondent’s mind - that’s roughly what we’re talking about.

Another future prediction is the shift to implicit prompting. Models will learn to understand our needs from our work context, without direct instructions. The system will see you’re working on a financial report and automatically adjust its style and knowledge base.

The prompt will stop being what you write and become what you do. This will be the era of “zero interface,” where the best prompt is no prompt. Our role will shift from direct operator to someone who merely sets the vector - the system performs all intermediate steps itself.



Instead of Conclusions: The Illusion of Control

So, where does that leave us? The era of “prompt hacking” is ending. It’s being replaced by “architectural empathy” - the ability to communicate with models in the language of their own structure. We’ve gone from trying to trick the system to trying to understand it, and that’s probably the most important paradigm shift.

Research confirms: there are no universal simple solutions. Politeness, threats, manipulation - none of it effectively works, though it varies somewhat between models, tasks, and even specific questions. There’s no philosopher’s stone, no user manual. The only thing that works consistently is structure that matches the model’s architecture.

And perhaps that’s the deepest meaning. Artificial intelligence doesn’t just give us answers - it forces us to learn to ask better questions. This often requires breaking complex problems into simple parts, clearly defining goals, structuring information. And by interacting with AI at this level, we inadvertently train our own systems thinking.

So I advise forgetting about finding the “perfect prompt” - it doesn’t exist. There’s only an endless process of iteration and gradually deeper understanding of how it all works: “The samurai has no goal, only the path.”

Interpret that however you want - because even this article isn’t about just getting answers.

Further Reading for the Architecturally-Minded

My conclusions are based not just on empirics and trial-and-error, but also on scientific research. So, if you want to dig deeper, these papers are the foundation for the arguments I’ve laid out here.

- **“Does Prompt Formatting Have Any Impact on LLM Performance?” (He et al., 2024)** and **“Towards LLMs Robustness to Changes in Prompt Format Styles” (2025)**
  These two papers provide the hard data behind my central thesis: format is not decoration, it’s architecture. The researchers empirically prove that seemingly minor, non-semantic changes in prompt structure - like switching from Markdown to JSON - can cause performance to swing wildly. This is the scientific validation for why XML-like structures work more consistently. The

work also confirms that larger models like GPT-4 are more robust to these format shifts than their predecessors, a point I raised earlier.

- **“Prompt Compression for Large Language Models: A Survey” (2024)**
  This survey offers an excellent technical overview of the future I touched on in the final section - the move away from natural language. It breaks down the concept of “soft prompts,” which are not written words but trainable, continuous vectors that the model processes more efficiently. This is the academic foundation for the idea of “prompt compression,” where the goal is no longer to write a perfect letter to the AI, but to send it a compressed bundle of thought.
- **“Unleashing the potential of prompt engineering for large language models” (2025)**
  This is a comprehensive review of the techniques that have defined prompt engineering, from the basic “few-shot” methods to more advanced reasoning frameworks like Tree of Thoughts (ToT) and Graph of Thoughts (GoT). It’s a good reference for understanding the academic landscape of the methods I argue are becoming obsolete, while also providing context for the non-linear reasoning models that are replacing them.
- **“PROMPT ENGINEERING AND THE EFFECTIVENESS OF LARGE LANGUAGE MODELS IN ENHANCING HUMAN PRODUCTIVITY” (2025)**
  This study closes the loop by connecting theory to real-world impact. Through user surveys, it demonstrates a direct correlation between the use of specific, structured prompts and perceived gains in productivity. It confirms that the efficiency we feel isn’t just a placebo; it’s the measurable result of a deliberate, architectural approach to communicating with AI, as an overwhelming majority of users agreed that clearer prompts lead to better results.

A message from our Founder

Hey, Sunil here. I wanted to take a moment to thank you for reading until the end and for being a part of this community.

Did you know that our team run these publications as a volunteer effort to over 3.5m monthly readers? **We don’t receive any funding, we do this to support the community.** ❤️

If you want to show some love, please take a moment to **follow me on LinkedIn, TikTok, Instagram**. You can also subscribe to our **weekly newsletter**.

And before you go, don’t forget to **clap** and **follow** the writer!